

UNIVERSIDAD POLITÉCNICA DEL VALLE DE TOLUCA

INGENIERIA EN TECNOLOGIAS DE LA INFORMACION E INNOVACION DIGITAL TSU* EN
DESRROLLO DE SOFTWARE MULTIPLATAFORMA

Estadía 1

Nombre del Proyecto: PLATAFORMA WEB INTEGRAL RESPONSIVA PARA LA GESTIÓN
AUTIMATIZADA DE CITAS EN LA OFICILIA 03 DEL REGISTRO CIVIL

P R E S E N T A.

Nombre del alumno: Heidy Paola Alvarez Hernandez

Nombre del asesor académico: Mtro. Pedro Valencia Perez

Nombre del asesor Externo: Lic. Cirilo Carmona Munguia

Almoloya de Juárez,

Contenido

Contenido	2
Introducción.....	4
Planteamiento del problema	5
Justificación del proyecto	5
Alcances	7
Limitaciones	8
Objetivo general	9
Objetivos específicos	9
Caracterización del área en la que se participo	10
Misión	11
Visión.....	11
Bases teórico prácticas utilizadas.....	13
Actividades realizadas.....	28
Recopilación de historias de usuario.....	28
Evidencia de la recopilación de las historias de usuarios	33
Requisitos funcionales	33
Requisitos no funcionales	36
Elección arquitectura para el software	37
Contribuciones realizadas	48
Productos del proyecto.....	48
Conclusiones.....	48
Recomendaciones.....	48



Referencias 49

Introducción

El presente documento describe el proceso de análisis, diseño y desarrollo de una plataforma web integral responsiva para la gestión automatizada de citas en la Oficialía 03 del Registro Civil de Villa Victoria, Estado de México. Su propósito es documentar de manera estructurada las actividades realizadas durante el desarrollo del proyecto, así como los fundamentos teóricos, metodológicos y técnicos que respaldan la solución propuesta.

En un primer apartado se presenta el planteamiento del problema, la justificación, los alcances, las limitaciones y los objetivos que orientan el desarrollo de la plataforma. Posteriormente, se describe la caracterización de la institución donde se llevó a cabo el proyecto, incluyendo su misión, visión y estructura organizacional, con el fin de contextualizar el entorno en el que se identificó la necesidad de implementar una herramienta tecnológica que contribuya a optimizar la atención a la ciudadanía.

También, se desarrolla el marco teórico, en el cual se abordan los conceptos, metodologías, tecnologías y estándares que sustentan el proyecto, tales como la ingeniería de software, las bases de datos, la metodología ágil Extreme Programming (XP), la programación orientada a objetos, Python, Django, PostgreSQL, la arquitectura de tres capas, el diseño responsivo y la tecnología de códigos QR, entre otros elementos fundamentales para el desarrollo de la plataforma.

De igual manera, se describen las actividades realizadas durante el desarrollo del software, desde la recopilación de historias de usuario y la definición de requisitos funcionales y no funcionales, hasta el diseño de la base de datos, la elaboración de las tarjetas CRC, la maquetación web, el desarrollo de la plataforma y la ejecución de pruebas para verificar su correcto funcionamiento.

Al final, el documento presenta los productos obtenidos como resultado del proyecto, así como las conclusiones y recomendaciones derivadas de la experiencia adquirida y del desarrollo de la plataforma, permitiendo evidenciar el cumplimiento de los objetivos planteados y la aplicación de los conocimientos adquiridos durante la formación académica.

Planteamiento del problema

El Registro Civil 03 de Villa Victoria es una institución encargada de registrar los actos más importantes de la vida de los ciudadanos, como nacimientos, matrimonios, divorcios, defunciones, entre otros. La demanda de atención es constante y, en temporadas altas como la inscripción en escuelas para primer ingreso o el registro de apoyos gubernamentales, se vuelve muy elevada.

Actualmente, todo el proceso de atención se maneja de forma manual y presencial, incluido el registro de citas. Por eso, la mayoría de los servicios se atienden de manera esporádica, sin ningún apoyo tecnológico que facilite la organización del trabajo.

El problema central es que no existe ninguna herramienta que permita a los ciudadanos agendar una cita desde casa. Para solicitar un trámite, la persona debe trasladarse físicamente a la oficina sin saber si habrá disponibilidad, cuánto tiempo tendrá que esperar ni cuánto dinero debe llevar. En muchos casos, el ciudadano pierde un día de trabajo para descubrir que ya no hay lugar y debe regresar otro día.

Dentro de la oficina, el personal solo cuenta con una agenda manual para organizar los trámites que requieren cita obligatoria. Los horarios se asignan a mano, lo que puede generar que se programen uno tras otro sin considerar el tiempo que dura cada trámite. Por ejemplo, el matrimonio a domicilio puede tomar hasta cuatro horas. Esta falta de control genera retrasos en cadena que afectan a todos los ciudadanos que llegan de manera esporádica.

Justificación del proyecto

La implementación de la plataforma de Gestión de Citas en el Registro Civil Municipal 03 de Villa Victoria beneficiará de forma directa cuatro procesos internos de la oficina que actualmente se realizan de manera manual y sin ningún apoyo tecnológico.

El primero es el proceso de registro y agendado de citas. Hoy este proceso depende de que el ciudadano se presente físicamente, de que el personal anote sus datos a mano y de que la asignación de horarios se haga sin ningún control sobre los tiempos ni sobre los empalmes.

Con la plataforma, este proceso pasa a ser digital, automatizado y validado desde el origen: los datos del ciudadano se capturan con formato obligatorio, el horario se asigna considerando la duración de cada trámite y la plataforma impide que dos citas ocupen el mismo espacio de tiempo. El proceso gana exactitud, orden y trazabilidad sin depender de la memoria ni del criterio del personal de turno.

Como segundo es el proceso de control de asistencia en ventanilla. Actualmente no existe ningún mecanismo formal para verificar que el ciudadano que llega a la ventanilla es el mismo que agendó la cita, ni para registrar su llegada de forma oficial. Con la plataforma, este proceso se formaliza mediante el escaneo del código QR que el ciudadano recibe al agendar, lo que cambia automáticamente el estado de la cita a asistida y deja constancia digital del momento exacto en que el ciudadano se presentó. Esto convierte un proceso informal en un proceso documentado y controlado.

El tercero es el proceso de registro de pagos y corte de caja. En la actualidad los pagos se anotan manualmente y el corte del día lo calcula el personal sin una herramienta que garantice la exactitud del resultado. Con la plataforma, cada pago queda vinculado directamente a la cita correspondiente y al tipo de trámite realizado, y el corte de caja se genera de forma automática al cierre del día con el desglose exacto por tipo de trámite. Este proceso deja de depender de cálculos manuales y pasa a tener un respaldo digital inmutable que puede consultarse en cualquier momento.

El cuarto es el proceso de gestión del catálogo de trámites. Hoy no existe un lugar centralizado donde estén registrados los trámites que ofrece la oficina, sus costos actualizados, los documentos que se requieren para cada uno y su estado de vigencia. Con la plataforma, el Administrador gestiona ese catálogo desde un panel digital, cualquier cambio de costo queda registrado automáticamente con fecha y usuario responsable, y la información siempre está disponible y actualizada tanto para el personal como para el ciudadano que consulta el portal. Este proceso pasa de ser informal y disperso a ser centralizado, documentado y controlado.

La actividad la plataforma no depende de cambios en la actitud del personal.

Alcances

La plataforma contará con un portal web público accesible sin necesidad de crear una cuenta de usuario, donde el ciudadano podrá consultar el catálogo de trámites disponibles con su costo actualizado y los documentos requeridos para cada uno, registrar sus datos personales con validación automática de CURP y Código Postal, seleccionar una fecha y hora disponible en un calendario interactivo que considera la duración de cada trámite, confirmar su cita y descargar el código QR que le servirá como comprobante al presentarse en la oficina.

El personal con rol de Oficial tendrá acceso a un panel protegido con usuario y contraseña donde podrá consultar la agenda de citas del día y días siguientes ordenadas por hora, gestionar la disponibilidad de horarios bloqueando o habilitando franjas de tiempo de forma semanal, registrar el pago del ciudadano al momento de su atención y consultar el monto recaudado del mes en curso con el historial de cortes de meses anteriores.

El personal con rol de Capturista podrá escanear el código QR del ciudadano al momento de su llegada a ventanilla para registrar su asistencia, lo que cambiará automáticamente el estado de la cita de pendiente a asistida.

El Administrador tendrá acceso completo la plataforma con capacidad para gestionar el catálogo de trámites creando, editando, activando y desactivando trámites con su nombre, duración, costo y documentos requeridos; gestionar las cuentas del personal asignando el rol correspondiente a cada usuario; consultar la bitácora de auditoría con filtros por usuario, fecha y tipo de acción; y generar y consultar los cortes de caja diarios y el historial mensual.

La plataforma registrará automáticamente todas las acciones críticas realizadas en cualquier módulo, cambios de estado, modificaciones de costo, registros de pago y accesos a la plataforma. Este registro será completamente inmutable y no podrá ser modificado ni eliminado por ningún rol.

La plataforma será desplegada en a Render, aprovechando el plan gratuito disponible por 90 días. Durante este periodo la plataforma estará accesible en línea a través de una URL pública generada por Render, con soporte automático de HTTPS que garantiza que las comunicaciones entre el usuario y el servidor estén cifradas. La base de datos PostgreSQL

también será gestionada dentro del plan gratuito de Render, que incluye almacenamiento suficiente para el volumen de datos esperado durante el periodo de pruebas.

Limitaciones

Dependencia de conectividad: La plataforma requiere una conexión a una red de área local o internet para funcionar, ya que al ser una aplicación web, no operará en modo desconectado (offline).

Hardware de captura: El correcto funcionamiento del módulo de validación depende de que el equipo utilizado cuente con una cámara web operativa y configurada correctamente. La plataforma no provee ni gestiona el hardware de lectura de códigos.

Gestión de datos de terceros: La plataforma no realizará validaciones automáticas de identidad con bases de datos externas del Registro Civil o entidades gubernamentales; su función se limita a procesar los registros previamente cargados en su propia base de datos.

Integridad de la información: La veracidad de los datos registrados depende exclusivamente de la captura manual del personal autorizado. La plataforma no incluye mecanismos de validación de documentos físicos, por lo que no puede asegurar que la información ingresada coincida con documentos reales si estos son capturados incorrectamente.

Seguridad de dispositivos: La seguridad final del acceso a la plataforma depende de la gestión de contraseñas de los usuarios y de la seguridad física de los equipos desde los cuales se accede, quedando fuera del alcance del software la protección contra intrusiones físicas o negligencia en el manejo de credenciales.

Volumen de almacenamiento: La plataforma está diseñada para un entorno de oficina estándar. El rendimiento de las gráficas y reportes podría verse afectado si el volumen de registros históricos alcanza cantidades masivas que excedan la capacidad de procesamiento del servidor donde se encuentre.

Es importante señalar que el plan gratuito de Render tiene algunas limitaciones que deben considerarse dentro del alcance del proyecto. El servidor puede entrar en modo de

suspensión tras un periodo de inactividad, lo que puede generar un tiempo de respuesta mayor en la primera petición después de una pausa. El almacenamiento y el tráfico están limitados a los recursos del nivel gratuito, por lo que la plataforma está dimensionada para operar en un entorno de pruebas y validación con el personal de la oficina durante los 90 días del plan gratuito, no para soportar una carga de producción masiva de forma indefinida. Una vez concluido ese periodo, la continuidad del servicio requerirá migrar a un plan de pago en Render o a un servidor propio de la institución según consideren los interesados en la plataforma.

Objetivo general

Desarrollar una plataforma web integral responsiva para el registro civil 03, orientada a la gestión automatizada de citas con validación mediante tecnología QR.

Objetivos específicos

Recopilar las historias de usuario para el software mediante la identificación y priorización de los requisitos funcionales y no funcionales de la plataforma de gestión de citas, considerando las necesidades operativas del personal de parte del oficial, administrador y capturista como los ciudadanos que acuden al Registro Civil 03 de Villa Victoria.

Elegir la arquitectura tecnológica más adecuada para el desarrollo del software, evaluando alternativas que garanticen escalabilidad, seguridad de la información y accesibilidad de la plataforma de agendamiento de citas para el Registro Civil 03 de Villa Victoria.

Crear la base de datos de la plataforma modelada en PlantUML que permita el almacenamiento, consulta y gestión segura de la información relacionada con los trámites de estado civil y las citas programadas en el Registro Civil 03 de Villa Victoria.

Diseñar las tarjetas CRC (Clase-Responsabilidad-Colaboración) para definir la estructura de clases, responsabilidades y colaboraciones de los componentes principales del software de gestión de citas.

Crear la maqueta del software que permita visualizar y validar el diseño de la experiencia de usuario para la plataforma de agendamiento y seguimiento de citas del Registro Civil 03 de Villa Victoria.

Codificar el backend y el frontend de la plataforma web para implementar la lógica del negocio, la interfaz de usuario y la integración entre ambos componentes de la plataforma de gestión de citas.

Realizar pruebas integrales y de estrés al software desarrollado con el fin de verificar su funcionalidad, rendimiento y estabilidad bajo diferentes escenarios de carga y uso concurrente.

Implementar las modificaciones y mejoras solicitadas durante las pruebas y revisiones técnicas para corregir errores, optimizar el desempeño y asegurar la calidad de la plataforma web.

Caracterización del área en la que se participo

El desarrollo del presente proyecto tuvo lugar en las instalaciones del Registro Civil de Villa Victoria, dependencia adscrita a Registro Civil 03 de Villa Victoria, situada en av. Lázaro Cardenas s/n, Col centro, Villa Victoria, Estado de Mexico. Esta institución forma parte del aparato administrativo del Estado y constituye uno de los organismos de mayor relevancia social, ya que tiene como función primordial dar fe pública de los actos que determinan el estado civil de las personas físicas dentro de su jurisdicción. Entre los actos que registra y certifica se encuentran los nacimientos, matrimonios, divorcios, defunciones, reconocimientos de hijos y adopciones, documentos que a su vez sirven como base para el ejercicio de derechos fundamentales de la ciudadanía, tales como la identidad legal, el acceso a servicios de salud, educación y seguridad social.

Desde su fundación, el Registro Civil 03 ha sido una institución de carácter permanente que opera bajo los lineamientos establecidos en la Constitución Política de los Estados Unidos Mexicanos, Código Civil del Estado de México y Reglamento del Registro civil del mismo estado, atendiendo de manera continua las necesidades de la población del municipio y sus alrededores. Su funcionamiento requiere de una coordinación constante entre las distintas áreas que lo integran, dado el volumen de trámites que se gestionan a diario y la responsabilidad legal que implica cada uno de los documentos que expide.

Misión

Garantizar a la población mexiquense certeza jurídica sobre los actos y hechos del estado civil de las personas, mediante servicios registrales accesibles, confiables, eficientes, transparentes y con pleno respeto a los derechos humanos, a través de la modernización, digitalización e innovación permanente de los procesos y servicios del Registro Civil del Estado de México.

Visión

Ser una institución registral líder a nivel nacional, reconocida por su innovación tecnológica, calidad en el servicio, cercanía con la ciudadanía, transparencia y eficiencia administrativa, consolidando un Registro Civil moderno, digital, interoperable y orientado a facilitar el acceso de las personas a su identidad y seguridad jurídica.

En cuanto a su estructura organizacional, el área se encuentra encabezada por el Oficial del Registro Civil, figura de autoridad responsable de validar y autorizar los actos registrales. Bajo su cargo se desempeña un equipo de trabajo conformado por dos personas, entre las que se encuentran el departamento de administración y auxiliar capturista, quienes en conjunto garantizan la operación diaria de la dependencia. Cada puesto cumple una función específica dentro del flujo de trabajo institucional, desde la recepción del ciudadano hasta la emisión y resguardo del documento oficial. Se presenta el organigrama en la Figura 1.

Figura 1

Organigrama del Registro Civil 03 de Villa Victoria



Nota. El organigrama presenta la estructura jerárquica de la Oficialía del Registro Civil 03 de Villa Victoria, integrada por el Oficial del Registro Civil como titular, el área de Administración y el Auxiliar Capturista.

En lo que respecta a la infraestructura tecnológica con la que cuenta la institución, se dispone de 2 equipos de cómputo, 2 impresoras y una copiadora para la cuestión de entregas de documentos oficiales y los procesos de atención al ciudadano se llevaban a cabo de forma manual en cuanto a la atención inmediata o registro de cita, lo que representa una de las principales áreas de oportunidad detectadas. Es precisamente en este contexto donde se identificó la problemática que da origen al presente proyecto, se detectó que el área de administración presentaba un descontrol en la gestión y seguimiento de la atención a los ciudadanos que acudían al Registro Civil de manera esporádica, situación que motivó el diseño y desarrollo de una solución acorde a las necesidades reales de la dependencia.

La participación activa dentro de esta institución permitió obtener un panorama amplio y detallado del funcionamiento interno del Registro Civil, así como comprender la importancia que tiene la modernización y optimización de sus procesos para brindar un servicio más eficiente, seguro y oportuno a la ciudadanía que día a día acude a sus instalaciones en busca de atender asuntos de carácter legal y personal.

Bases teórico prácticas utilizadas

El desarrollo de una plataforma web es un proceso que va mucho más allá de escribir código. Requiere entender qué herramientas se usarán, con qué metodología se trabajará, qué estándares garantizarán su calidad y sobre qué bases técnicas se apoyará. El presente marco teórico tiene el propósito de presentar los conceptos que fundamentan la construcción de la plataforma propuesta.

En este sentido, antes de construir, se necesita pensar. Y para pensar bien, se necesitan bases teóricas sólidas. Eso es exactamente lo que este marco teórico ofrece.

La ingeniería de software es la disciplina que se encarga de aplicar métodos, herramientas y principios de ingeniería al desarrollo del software. Esto significa que no basta con que una persona sepa programar: hace falta planear, diseñar, construir, probar y mantener el sistema de manera organizada para que funcione bien, dure en el tiempo y pueda modificarse sin romperse. Briano (2023) explica que la ingeniería de software nació como respuesta a la "crisis del software", un fenómeno que ocurrió en las décadas de 1960 y 1970, cuando los proyectos de software constantemente se retrasaban, costaban más de lo previsto o simplemente no funcionaban como se esperaba. Ante esto, fue necesario crear una disciplina que diera orden y rigor al proceso de construir software, de la misma manera en que la ingeniería civil lo hace con los puentes o los edificios. Según el autor, la ingeniería de software busca que el software sea un producto confiable, eficiente y que cumpla con lo que el usuario necesita.

Para el presente proyecto, la ingeniería de software es el marco general dentro del cual se toman todas las decisiones: qué metodología seguir, cómo modelar el sistema, cómo dividirlo en partes manejables y cómo garantizar que cada parte funcione correctamente. En efecto, sin este marco, el desarrollo de la plataforma sería improvisado y sus resultados serían impredecibles.

Una plataforma web es un sistema de software que funciona a través de un navegador de internet. Morales (2021) define una plataforma web como "un entorno digital integrado que permite a los usuarios interactuar con servicios, contenidos y funciones a través de internet, sin necesidad de instalar software adicional en sus dispositivos" (p. 14). A diferencia de las

aplicaciones de escritorio que hay que instalar en cada computadora, una plataforma web solo requiere que el usuario abra su navegador y entre a la dirección correcta. El sistema vive en un servidor y todos los usuarios acceden a él de manera centralizada.

Montilva y Barrios (2022) señalan que la ingeniería del software se distingue de otras actividades de programación porque "identifica, describe y relaciona todos aquellos conceptos necesarios para comprender los fundamentos de esta ingeniería, y le da importancia a los procesos que la distinguen como una actividad de ingeniería propiamente dicha". Dicho de otro modo, la ingeniería de software no solo se trata de escribir código, sino de entender el problema, diseñar la solución, construirla con orden y verificar que funciona.

Por lo tanto, la plataforma web que se desarrolla en este proyecto sigue los principios de la ingeniería de software: tiene un proceso definido, usa herramientas apropiadas, aplica una metodología reconocida y busca cumplir con estándares de calidad. Cada sección de este marco teórico es, en realidad, una parte de ese proceso.

Una base de datos es un lugar donde se guarda información de forma organizada para que pueda buscarse, consultarse y actualizarse fácilmente. Piénsese en una agenda de contactos, pero mucho más grande y capaz de manejar miles de datos al mismo tiempo, con muchos usuarios usando el sistema a la vez.

Las bases de datos relacionales son las más usadas en el desarrollo de plataformas web. En ellas, la información se guarda en tablas parecidas a las hojas de cálculo donde cada fila es un registro y cada columna es un tipo de dato. Lo que las hace poderosas es que se pueden relacionar unas tablas con otras. Por ejemplo, una tabla de usuarios puede relacionarse con una tabla de registros, de modo que el sistema sabe qué usuario hizo qué registro. Briano (2023) indica que los sistemas de gestión de bases de datos son los programas que permiten crear, consultar y administrar estas bases de datos. Estos sistemas reciben instrucciones escritas en un lenguaje llamado SQL, que es el estándar para trabajar con bases de datos relacionales.

PostgreSQL es uno de los sistemas de gestión de bases de datos más completos y confiables que existen. Es de código abierto, lo que significa que es gratuito y que miles de personas en el mundo lo mejoran constantemente. Tiene más de 35 años de desarrollo activo y es usado por empresas grandes y pequeñas en todo el mundo.

PostgreSQL Global Development Group (2024) lo describe como un sistema "objeto-relacional de código abierto con una sólida reputación por su confiabilidad, robustez de características y rendimiento". En otras palabras, es una herramienta que funciona bien, que no falla fácilmente y que puede manejar grandes cantidades de información sin problema.

Para este proyecto, PostgreSQL fue elegido porque se conecta de manera natural con Django, el framework que se usa para construir la plataforma. Además, soporta las llamadas transacciones ACID, que garantizan que si algo falla en medio de un proceso por ejemplo, si se va la luz mientras se guarda un dato, el sistema no quede en un estado inconsistente. En este sentido, PostgreSQL protege la integridad de la información.

El conjunto formado por Python, Django y PostgreSQL es uno de los más populares para el desarrollo web moderno, precisamente porque cada uno complementa al otro: Python da el lenguaje, Django da la estructura del sistema, y PostgreSQL guarda los datos de forma segura.

Antes de empezar a escribir código, es importante tener claro cómo va a funcionar el sistema. Para eso se usan diagramas: representaciones visuales que muestran qué partes tiene el sistema, cómo se relacionan entre sí y cómo fluye la información. Estos diagramas hacen que todos los miembros del equipo programadores, diseñadores y el cliente hablen el mismo idioma.

PlantUML es una herramienta de código abierto que permite crear estos diagramas escribiendo texto sencillo, sin necesidad de arrastrar figuras o usar programas complicados. Como explica su documentación oficial (PlantUML, 2025), la herramienta genera de manera automática diagramas de clases, de casos de uso, de secuencia y muchos otros tipos, a partir de una descripción textual simple. Esto hace que los diagramas sean fáciles de actualizar cada vez que el diseño del sistema cambia.

En este proyecto, PlantUML fue usado para crear los diagramas que muestran cómo se organiza la plataforma: qué clases existen, qué hace cada una, cómo interactúan el usuario y el sistema, y cómo fluye la información desde que el usuario hace una solicitud hasta que recibe una respuesta. Así mismo, estos diagramas sirvieron como guía durante la construcción de la plataforma, reduciendo errores y malentendidos.

PlantUML se conecta directamente con la siguiente etapa del proceso: el diseño dentro de la metodología XP, específicamente con las tarjetas CRC (Clase, Responsabilidad, Colaboración), que son la versión en papel de lo que los diagramas muestran de forma visual. Por consiguiente, PlantUML y las tarjetas CRC trabajan juntos para dar claridad antes de que empiece la codificación.

Una metodología de desarrollo de software es una forma de organizar el trabajo del equipo para construir el sistema. Existen dos grandes tipos: las metodologías tradicionales, que planean todo desde el principio y no les gustan los cambios; y las metodologías ágiles, que aceptan que los requisitos cambian y se adaptan a eso. Letelier y Penadés (2006), investigadores de la Universidad Politécnica de Valencia, explican que las metodologías ágiles dan mayor valor "al individuo, a la colaboración con el cliente y al desarrollo incremental del software con iteraciones muy cortas". Esto significa que en lugar de planear todo durante meses y luego construir todo de una vez, las metodologías ágiles dividen el trabajo en ciclos cortos, entregan partes funcionales del sistema con frecuencia y ajustan el rumbo según lo que el cliente necesita.

Las metodologías ágiles son especialmente adecuadas cuando los requisitos no están completamente claros desde el principio, como suele ocurrir en el desarrollo de plataformas web. En este tipo de proyectos, el cliente descubre lo que realmente necesita a medida que ve el sistema funcionar, y la metodología ágil permite incorporar esos cambios sin que el proyecto colapse.

La Programación Extrema, conocida como XP por sus siglas en inglés (eXtreme Programming), es una metodología ágil creada por Kent Beck a finales de los años noventa. Es una de las más populares entre los equipos pequeños y medianos porque sus reglas son claras, sus prácticas son concretas y pone mucha atención en la calidad del código y en la satisfacción del cliente.

Letelier y Penadés (2006) describen XP como "una metodología ágil centrada en potenciar las relaciones interpersonales como clave para el éxito en desarrollo de software, promoviendo el trabajo en equipo, preocupándose por el aprendizaje de los desarrolladores, y propiciando un buen clima de trabajo". En otras palabras, XP entiende que los sistemas los construyen personas, y que si las personas trabajan bien juntas y con claridad, el resultado será mejor.

XP se apoya en cuatro valores fundamentales: la comunicación constante entre todos los miembros del equipo y con el cliente; la simplicidad, que significa hacer lo necesario y no más; la retroalimentación continua para corregir a tiempo; y el respeto entre todos los participantes del proyecto. Estos valores se materializan en cuatro etapas concretas que se describen a continuación.

La planeación en XP se basa en las historias de usuario, que son descripciones breves y en lenguaje natural de las funcionalidades que el sistema debe proporcionar desde la perspectiva del usuario final. Cada historia de usuario incluye quién realiza la acción, qué acción desea realizar y con qué propósito. Este formato garantiza que el desarrollo permanezca centrado en las necesidades reales del usuario.

Según Beck y Andres (2004), las historias de usuario son la unidad básica de planificación en XP y permiten estimar el esfuerzo necesario para implementar cada funcionalidad. En el contexto de la plataforma web de este proyecto, se elaboraron historias de usuario para cada módulo funcional, incluyendo el registro de usuarios, la consulta de información y la generación de reportes con código QR.

No cabe duda de que las historias de usuario representan una herramienta poderosa para alinear las expectativas del cliente con las capacidades del equipo de desarrollo, ya que permiten priorizar las funcionalidades de mayor valor y planificar las iteraciones de desarrollo de forma realista y flexible.

La etapa de diseño en XP comprende dos herramientas fundamentales: la maquetación y las tarjetas CRC (Clase, Responsabilidad, Colaboración). La maquetación, o wireframing, consiste en la representación visual de las pantallas del sistema sin incluir elementos gráficos definitivos, lo que permite validar la estructura y navegación de la plataforma antes de invertir recursos en el diseño final.

Por su parte, las tarjetas CRC son una técnica de diseño orientado a objetos que permite identificar las clases del sistema, sus responsabilidades y sus colaboraciones con otras clases. Cada tarjeta representa una clase y contiene su nombre, la lista de responsabilidades que debe cumplir y las clases con las que colabora para cumplirlas. Esta herramienta fue utilizada en el presente proyecto para definir la arquitectura de clases de la plataforma web.

En este sentido, Pressman y Maxim (2014) señalan que el diseño en el contexto de las metodologías ágiles no implica una especificación detallada y exhaustiva desde el inicio, sino que evoluciona de forma iterativa a medida que el sistema se desarrolla y los requisitos se clarifican. Esto permite que la arquitectura de la plataforma sea flexible y pueda adaptarse a los cambios sin incurrir en costos excesivos de refactorización.

La etapa de codificación en XP se caracteriza por prácticas como la programación en pares, la integración continua y el desarrollo guiado por pruebas (TDD). En el contexto del presente proyecto, la codificación se dividió en dos grandes componentes: el frontend y el backend.

El frontend comprende todos los elementos visuales e interactivos que el usuario percibe directamente en su navegador, incluyendo las páginas HTML, los estilos CSS y los scripts JavaScript. Para garantizar la coherencia visual y la accesibilidad de la plataforma, se adoptaron las guías de identidad gráfica del Gobierno de México, que establecen estándares de diseño, tipografía, colores y componentes de interfaz de usuario para sitios web gubernamentales (Gobierno de México, 2024).

El backend, por su parte, comprende la lógica de negocio, el acceso a la base de datos y la generación de respuestas para las solicitudes del cliente. En este proyecto, el backend fue implementado con el framework Django, que proporciona una arquitectura robusta y segura para el desarrollo de aplicaciones web en Python.

Las pruebas son un componente fundamental de XP. Beck y Andres (2004) establecen que en XP las pruebas se escriben antes del código (Test-Driven Development), lo que garantiza que cada funcionalidad implementada cumple con los criterios de aceptación definidos en las historias de usuario.

Para el presente proyecto se realizaron dos tipos principales de pruebas: pruebas integrales y pruebas de estrés. Las pruebas integrales verifican que los diferentes módulos de la plataforma funcionen correctamente cuando trabajan conjuntamente, simulando escenarios de uso real. Pressman y Maxim (2014) definen las pruebas de integración como el proceso de verificar que los componentes de un sistema combinados producen los resultados esperados.

Las pruebas de estrés, por su parte, evalúan el comportamiento de la plataforma bajo condiciones de carga extrema, identificando cuellos de botella y garantizando que el sistema pueda soportar el número esperado de usuarios concurrentes sin degradar su rendimiento. Esto resulta especialmente importante para una plataforma web de acceso público, donde la demanda puede ser impredecible y variable.

Para este proyecto se realizarán dos tipos de pruebas. Las pruebas integrales verifican que las diferentes partes del sistema funcionen bien cuando trabajan juntas. Como explica Briano (2023), "la integración de componentes es uno de los momentos más críticos del desarrollo, ya que es cuando se descubre si los módulos se comunican correctamente entre sí". En la práctica, estas pruebas simulaban el uso real de la plataforma: un usuario se registra, llena un formulario, guarda la información y genera un reporte. Si todo funciona de principio a fin, la prueba integral fue exitosa.

Las pruebas de estrés, por su parte, evalúan cómo se comporta la plataforma cuando muchas personas la usan al mismo tiempo. Estas pruebas son importantes porque una plataforma web gubernamental puede recibir muchas visitas simultáneas, especialmente en días de alta demanda. Las pruebas de estrés permiten identificar en qué momento el sistema empieza a ser lento o a fallar, para poder corregirlo antes de que llegue a los usuarios reales.

Las WCAG 2.1 son las Pautas de Accesibilidad para el Contenido Web, publicadas en 2018 por el Consorcio World Wide Web (W3C). Son el estándar internacional que define qué debe cumplir un sitio web para que puedan usarlo personas con distintas capacidades: personas que no pueden ver bien, personas que usan el teclado en lugar del ratón, personas con dificultades cognitivas, entre otras.

El W3C (2018) establece que las WCAG 2.1 están organizadas alrededor de cuatro principios: la información debe ser perceptible (que el usuario pueda verla o escucharla), las funciones deben ser operables (que se puedan usar de distintas formas), el contenido debe ser comprensible (que el usuario entienda qué hace el sistema), y el sistema debe ser robusto (que funcione bien con distintos navegadores y tecnologías de apoyo).

Para la plataforma de este proyecto, se cumplieron los criterios de nivel AA de las WCAG 2.1, que es el nivel recomendado para sitios web de uso general. Entre las medidas concretas que se aplicaron están: que los textos tengan suficiente contraste con el fondo para

que puedan leerse bien, que todos los campos de los formularios tengan etiquetas claras, que la plataforma pueda usarse solo con el teclado, y que las imágenes tengan texto alternativo para quienes usen lectores de pantalla.

El cumplimiento de las WCAG 2.1 no es solo una buena práctica: en el contexto de una plataforma gubernamental mexicana, garantizar la accesibilidad es también una responsabilidad hacia todos los ciudadanos que necesitarán usarla, independientemente de sus capacidades físicas o del dispositivo que tengan disponible.

La norma ISO 9241-11:2018 es un estándar internacional que define qué significa que un sistema sea usable. Publicada por la Organización Internacional de Normalización en 2018, esta norma dice que un sistema es usable cuando las personas a las que está dirigido pueden alcanzar sus objetivos de manera efectiva, eficiente y con satisfacción.

En concreto, la ISO 9241-11:2018 define tres componentes de la usabilidad. La efectividad es el grado en que el usuario logra lo que quiere lograr. La eficiencia es qué tan rápido y con qué esfuerzo lo logra. La satisfacción es cómo se siente el usuario mientras usa el sistema: si le resulta cómodo, claro y agradable de usar.

Para este proyecto, los tres componentes de la usabilidad guiaron las decisiones de diseño. Se buscó que la plataforma fuera efectiva (que los usuarios pudieran completar sus tareas), eficiente (que no tuvieran que hacer demasiados clics o llenar formularios innecesariamente complicados) y satisfactoria (que la experiencia fuera clara y agradable).

La ISO 9241 y las WCAG 2.1 son complementarias: una se enfoca en la accesibilidad (que todos puedan usar el sistema) y la otra en la usabilidad (que todos puedan usarlo bien). Juntas, aseguran que la plataforma esté diseñada para las personas, no solo para la tecnología.

Los objetivos SMART son una herramienta para formular metas de forma que sean claras y alcanzables. La palabra SMART es un acrónimo que en español significa: Específico, Medible, Alcanzable, Relevante y con límite de Tiempo. Aplicar este enfoque al proyecto significa que cada objetivo que se plantea es concreto, se puede verificar si se cumplió o no, es posible de lograr con los recursos disponibles, tiene sentido dentro del proyecto y tiene una fecha de cumplimiento.

Por ejemplo, en lugar de decir "mejorar la accesibilidad de la plataforma" que es vago y no se puede medir se puede decir "lograr que el 100% de las páginas de la plataforma cumplan con los criterios de nivel AA de las WCAG 2.1 antes de la entrega final del proyecto". Este objetivo es específico, medible (se puede auditar), alcanzable (hay herramientas automáticas que lo verifican), relevante para el proyecto y tiene una fecha clara.

Los objetivos SMART conectan directamente con las pruebas de la metodología XP: si el objetivo está bien definido, se puede saber exactamente qué probar para verificar que se cumplió. De esta forma, los estándares de calidad no son solo requisitos abstractos, sino metas concretas que guían el desarrollo desde el principio hasta el final.

La programación orientada a objetos, conocida como POO, es una forma de organizar el código de un programa. En lugar de escribir instrucciones una tras otra, la POO organiza el sistema en "objetos": representaciones de las cosas del mundo real que el sistema maneja. Cada objeto tiene datos (lo que sabe de sí mismo) y comportamientos (lo que puede hacer).

Por ejemplo, en la plataforma de este proyecto existe un objeto "usuario". Ese objeto sabe su nombre, su correo y su contraseña (esos son sus datos). También puede iniciar sesión, cerrar sesión y editar su perfil (esos son sus comportamientos). De la misma forma, existe un objeto "reporte" que sabe qué información contiene y puede generarse en formato PDF o con código QR.

La POO se apoya en cuatro principios que la hacen especialmente útil para sistemas grandes y complejos como una plataforma web. La encapsulación significa que cada objeto esconde sus detalles internos y solo muestra al exterior lo que es necesario. La herencia permite que un tipo de objeto comparta características con otro sin tener que repetir el código. El polimorfismo permite que diferentes objetos respondan de manera distinta al mismo tipo de instrucción. Y la abstracción consiste en representar solo los aspectos del mundo real que son relevantes para el sistema, ignorando los detalles que no importan.

Briano (2023) señala que la POO "hace que los sistemas sean más fáciles de entender, de modificar y de mantener, ya que cada cambio se hace en el objeto que corresponde sin afectar al resto del sistema". Esto es especialmente importante en el contexto del desarrollo

ágil, donde los cambios son frecuentes. Si el sistema está bien organizado en objetos, agregar una nueva funcionalidad o corregir un error es más sencillo y menos riesgoso.

En la plataforma de este proyecto, la POO se aplica a través de Python y Django. Cada entidad del sistema usuario, registro, reporte, módulo es una clase. Las clases definen cómo son esos elementos y qué pueden hacer. Django, por su parte, usa estas clases directamente para comunicarse con la base de datos, de modo que el código que el programador escribe en Python se traduce automáticamente en tablas y registros en PostgreSQL.

Python es un lenguaje de programación creado en 1991 por Guido van Rossum. Hoy en día es uno de los lenguajes más populares del mundo, usado en desarrollo web, análisis de datos, inteligencia artificial y automatización. Su gran ventaja es que su sintaxis es parecida al lenguaje natural: el código de Python es fácil de leer incluso para personas que no programan.

La Python Software Foundation (2024) describe el lenguaje como "fácil de aprender, potente y con un enfoque simple pero eficaz para la programación orientada a objetos". Estas características lo hacen ideal para el desarrollo de plataformas web, donde se necesita escribir código claro, que funcione bien y que sea fácil de mantener con el tiempo.

Para este proyecto, se usó Python 3, que es la versión actual del lenguaje. Python es la base sobre la que funciona Django: cuando el servidor recibe una solicitud del usuario, es Python el que la procesa, aplica las reglas del negocio, consulta la base de datos y devuelve la respuesta. En otras palabras, Python es el motor que hace funcionar todo el backend de la plataforma.

Python también se conecta con PostgreSQL a través del ORM de Django, con las herramientas de prueba automatizada, con la generación de reportes en PDF y con la creación de códigos QR. En este sentido, Python es el hilo que une todas las partes técnicas del proyecto.

Visual Studio Code, conocido como VS Code, es un editor de código desarrollado por Microsoft. Es gratuito, funciona en Windows, Mac y Linux, y es uno de los más usados por desarrolladores web en todo el mundo. Su popularidad se debe a que es ligero, rápido y tiene una enorme cantidad de extensiones que lo hacen más potente.

Para este proyecto, VS Code fue el entorno principal de desarrollo. Se usaron extensiones específicas para Python y Django que ofrecen funciones como el autocompletado de código (el editor sugiere cómo terminar una línea antes de que el programador la complete), el resaltado de errores en tiempo real, la integración con Git para el control de versiones, y la posibilidad de ver los diagramas de PlantUML directamente en el editor.

VS Code conecta todas las herramientas del proyecto en un solo lugar: el programador puede escribir código en Python, ver los diagramas de PlantUML, conectarse a la base de datos PostgreSQL y subir los cambios al repositorio, todo desde la misma ventana. Esto reduce el tiempo que se pierde cambiando de una herramienta a otra y permite que el equipo trabaje de forma más fluida.

Un framework es un conjunto de herramientas y reglas que simplifica el desarrollo de un tipo específico de software. En lugar de construir todo desde cero, el framework ofrece piezas ya hechas que el programador puede usar y personalizar. Para construir aplicaciones web con Python, el framework más popular es Django.

Django fue creado en 2003 y liberado como software libre en 2005. La Django Software Foundation (2024) lo define como "un framework web de alto nivel para Python que fomenta el desarrollo rápido y el diseño limpio y pragmático". Esto significa que Django permite construir aplicaciones web de manera rápida porque ya tiene resueltos muchos de los problemas comunes: cómo manejar usuarios y contraseñas, cómo conectarse a la base de datos, cómo mostrar páginas web, cómo proteger el sistema de ataques frecuentes.

Django usa un patrón de diseño llamado MVT, que significa Modelo, Vista y Plantilla. El Modelo define la estructura de los datos y se comunica con la base de datos. La Vista contiene la lógica: recibe la petición del usuario, consulta el modelo y decide qué mostrar. La Plantilla define cómo se ve la página que el usuario recibe en su navegador. Este patrón mantiene ordenado el código y hace que cada parte del sistema tenga una responsabilidad clara.

Entre las cosas que Django hace de forma automática y que ahorran mucho trabajo en este proyecto están: la gestión de usuarios con roles y permisos diferenciados, la protección contra los ataques web más comunes (como la inyección SQL o el robo de sesiones), el panel de administración que permite gestionar los datos sin escribir código adicional, y la integración directa con PostgreSQL a través de su ORM.

El ORM de Django, que significa "mapeador objeto-relacional", es una de sus herramientas más útiles. Permite que el programador trabaje con la base de datos usando código Python, sin necesidad de escribir instrucciones SQL directamente. El programador define los modelos como clases de Python, y Django se encarga de crear las tablas en PostgreSQL y de traducir las operaciones al lenguaje que la base de datos entiende. Esto hace el código más limpio, más fácil de leer y menos propenso a errores.

Cuando se construye una plataforma web, es necesario decidir cómo se va a organizar internamente. Esta decisión se llama arquitectura del software. Para este proyecto se eligió la arquitectura monolítica de tres capas, que es una de las más usadas en el desarrollo web y que encaja de manera natural con Django.

La palabra "monolítica" significa que todo el sistema vive en un solo lugar: un único proyecto que contiene todas sus partes. Esto simplifica el desarrollo y el despliegue, especialmente en proyectos que no necesitan escalar a millones de usuarios. Las tres capas son divisiones lógicas dentro de ese sistema único, que separan las responsabilidades para mantener el código organizado.

La primera capa es la de presentación, que corresponde al frontend: todo lo que el usuario ve en su navegador. Aquí están las páginas HTML, los estilos CSS y los scripts que hacen que la interfaz sea interactiva. Esta capa recibe las acciones del usuario (hacer clic en un botón, llenar un formulario) y las envía a la capa siguiente.

La segunda capa es la de lógica de negocio, que en Django corresponde a las vistas. Aquí se procesan las peticiones del usuario: se verifica si tiene permiso para hacer lo que quiere hacer, se aplican las reglas del sistema, se calcula lo que hay que calcular y se decide qué información enviar de vuelta. Esta capa actúa como intermediaria entre lo que el usuario ve y los datos que están guardados.

La tercera capa es la de datos, que corresponde a los modelos de Django y a la base de datos PostgreSQL. Aquí se guarda, se consulta y se actualiza toda la información que el sistema maneja. Esta capa solo recibe instrucciones de la capa de lógica de negocio: nunca se comunica directamente con el usuario.

Esta separación en tres capas tiene una ventaja práctica muy importante: si se necesita cambiar algo en una capa, no es necesario tocar las otras. Por ejemplo, si se quiere cambiar el diseño visual de la plataforma, solo se modifica la capa de presentación, sin tocar la lógica ni los datos. Esto hace que el mantenimiento sea mucho más sencillo.

Una plataforma web es un sistema de software que funciona a través de un navegador de internet. A diferencia de las aplicaciones de escritorio que hay que instalar en cada computadora, una plataforma web solo requiere que el usuario abra su navegador y entre a la dirección correcta. El sistema vive en un servidor y todos los usuarios acceden a él de manera centralizada.

Las plataformas web tienen grandes ventajas para proyectos de uso institucional o gubernamental: no requieren instalación en los dispositivos de los usuarios, se actualizan en un solo lugar y los cambios llegan a todos de inmediato, pueden usarse desde cualquier dispositivo con internet (computadora, tableta o teléfono), y permiten que múltiples personas trabajen con el mismo sistema al mismo tiempo.

La plataforma web de este proyecto fue diseñada para que usuarios con diferentes roles puedan gestionar información, consultar registros, generar reportes con código QR y administrar el sistema. Todo esto desde su navegador, sin necesidad de instalar ningún programa adicional.

Para que la plataforma tenga una apariencia oficial y coherente con otros sistemas del Gobierno de México, se siguieron las guías de identidad gráfica publicadas en el portal GOB.MX. Estas guías definen exactamente cómo deben verse los sitios web gubernamentales: qué fuentes tipográficas usar, qué colores, cómo deben lucir los botones, cómo se organiza la navegación y qué iconografía es apropiada.

El Gobierno de México (2024a) establece en su guía de identidad gráfica digital los estándares de diseño que deben seguir todos los sitios oficiales. Esto incluye el uso de la tipografía Montserrat para títulos y Roboto para textos corridos, una paleta de colores institucionales basada en el verde gubernamental, y componentes de interfaz coherentes con la identidad visual del gobierno federal.

En cuanto a los botones, la guía de la versión 3 del sistema GOB.MX (Gobierno de México, 2024c) establece una jerarquía visual clara: los botones primarios se usan para las acciones más importantes (como "guardar" o "enviar"), los botones secundarios para acciones alternativas, y los botones de alerta para acciones que pueden tener consecuencias difíciles de revertir (como "eliminar"). Esta jerarquía guía al usuario de forma intuitiva hacia las acciones que se esperan de él.

El uso de estas guías no solo da a la plataforma una apariencia profesional e institucional: también contribuye a la usabilidad, porque los ciudadanos que ya usan otros servicios del gobierno encontrarán una experiencia coherente y familiar. Desde esta perspectiva, las guías de identidad gráfica son también un estándar de calidad.

El diseño responsivo es la capacidad de un sitio web de adaptarse automáticamente al tamaño de la pantalla del dispositivo desde el que se accede. Esto significa que la misma plataforma se verá bien en una pantalla grande de escritorio, en una tableta y en un teléfono celular, sin que el usuario tenga que hacer nada especial.

Esta tecnología es hoy una necesidad básica. En México y en el mundo, la mayoría de las personas acceden a internet principalmente desde sus teléfonos celulares. Si una plataforma web no funciona bien en dispositivos móviles, una gran parte de los usuarios potenciales simplemente no podrá usarla bien, o dejará de intentarlo.

El diseño responsivo se implementa mediante reglas de CSS llamadas "media queries", que le dicen al navegador cómo debe mostrar el contenido según el tamaño de la pantalla. Por ejemplo, en una pantalla grande los elementos se muestran en varias columnas; en una pantalla de teléfono, esos mismos elementos se reorganizan en una sola columna para que quepan bien y sean fáciles de tocar con el dedo.

Marcotte (2015), quien introdujo y popularizó el concepto de diseño responsivo, explica que "el diseño responsivo es pensar más allá de la pantalla de escritorio y crear diseños que se adapten a las necesidades del usuario, sin importar cuán grande o pequeña sea su pantalla". Para la plataforma de este proyecto, se definieron tres puntos de adaptación: para pantallas de teléfonos (menos de 768 píxeles), para tabletas (entre 768 y 1024 píxeles) y para pantallas de escritorio (más de 1024 píxeles).

El diseño responsivo conecta directamente con los estándares de calidad descritos anteriormente. Las WCAG 2.1, por ejemplo, incluyen criterios específicos sobre cómo debe adaptarse el contenido a diferentes tamaños de pantalla y orientaciones. En este sentido, cumplir con el diseño responsivo es también cumplir con los requisitos de accesibilidad.

Un código QR (que en inglés significa "Quick Response", es decir, "respuesta rápida") es un tipo de código de barras bidimensional que puede almacenar mucha más información que los códigos de barras tradicionales. Con la cámara de un teléfono, cualquier persona puede escanear un código QR y acceder instantáneamente a la información que contiene.

Los códigos QR se usan hoy en casi todos los ámbitos: en restaurantes para ver el menú, en museos para obtener información sobre una obra, en documentos oficiales para verificar su autenticidad, y en productos para conocer su procedencia. Su gran ventaja es que conectan el mundo físico con el mundo digital de manera rápida y sin fricción: el usuario no tiene que escribir ninguna dirección ni buscar ningún archivo.

El estándar que define las especificaciones técnicas del código QR es la norma ISO/IEC 18004, cuya versión más reciente fue publicada en 2024. Esta norma establece cómo debe construirse el código, cómo se codifica la información y qué niveles de corrección de errores existen (ISO/IEC, 2024). Los códigos QR pueden seguir funcionando aunque una parte del símbolo esté dañada u ocluida, gracias a su sistema de corrección de errores.

Para la plataforma de este proyecto, los códigos QR se usan en los reportes generados por el sistema. Cada reporte incluye un código QR que, al escanearse con un teléfono, lleva directamente al registro correspondiente en la plataforma, permitiendo verificar su autenticidad y consultar información actualizada en tiempo real. Esto agrega valor al documento físico y reduce la posibilidad de falsificación.

Una vez que la plataforma web está construida, necesita estar disponible en internet para que los usuarios puedan acceder a ella. Esto se llama "despliegue" o "puesta en producción". Para hacerlo, se necesita un servidor: una computadora que esté encendida todo el tiempo y que atienda las peticiones de los usuarios.

Hoy en día, la mayoría de las aplicaciones web se despliegan en la nube, usando plataformas especializadas que ofrecen servidores como un servicio. Esto tiene varias ventajas:

no hay que comprar ni mantener hardware propio, el sistema puede crecer (o reducirse) según la demanda, y hay garantías de disponibilidad y seguridad que sería muy difícil lograr con un servidor propio.

Para este proyecto se eligió Render como plataforma de despliegue. Render (2024) es una plataforma en la nube que permite desplegar aplicaciones web, bases de datos y servicios de fondo de manera sencilla, con "certificados TLS gratuitos, una red de distribución global, protección contra ataques DDoS, redes privadas y despliegues automáticos desde Git". En términos simples, Render se encarga de la infraestructura para que el equipo de desarrollo pueda concentrarse en el sistema.

Una de las características más útiles de Render para este proyecto es que se conecta directamente con el repositorio de código en GitHub. Cada vez que el equipo sube un cambio al repositorio, Render detecta la actualización y despliega automáticamente la nueva versión de la plataforma. Esto hace que el proceso de actualización sea rápido, sin interrupciones para los usuarios, y elimina la necesidad de intervención manual.

Render también ofrece soporte nativo para Django y PostgreSQL, las dos tecnologías principales de este proyecto. Esto significa que la configuración del servidor es mínima: la plataforma funciona en producción casi de la misma manera en que funciona durante el desarrollo. Esta coherencia reduce los errores que suelen ocurrir cuando se pasa de un entorno de prueba a un entorno real.

Actividades realizadas

Recopilación de historias de usuario

En esta fase se trabajó directamente con el personal del Registro Civil para identificar las necesidades reales de cada rol: el oficial, el capturista y el administrador. Cada necesidad se documentó como una historia de usuario siguiendo el formato de XP, especificando quién la solicitaba, qué quería lograr y bajo qué criterios se consideraría completada. El resultado fue un conjunto de ocho historias de usuario, a continuación, se muestran las ocho tablas de las historias de usuario.

Tabla 1

Historia de usuario 1

Historia de Usuario	
Número: 1	Usuario: Oficial del Registro Civil
Nombre historia: Registro de Solicitud de Cita	
Prioridad en negocio:	Riesgo en desarrollo:
Alta	Media
Puntos estimados: 5	Iteración asignada: 1
Programador responsable: Heidy Paola Alvarez Hernandez	
Descripción:	
El ciudadano accede al portal web sin necesidad de crear una cuenta. Captura su nombre completo, CURP, Código Postal y elige el día y la hora de su trámite en un calendario interactivo que muestra solo los horarios disponibles.	

La plataforma agenda el espacio automáticamente considerando la duración de cada trámite. Si dos ciudadanos intentan elegir el mismo horario al mismo tiempo, la plataforma rechaza y le pide que elija otro horario disponible. El Oficial puede modificar los horarios disponibles semanalmente o de forma inmediata ante cualquier contratiempo.

Observaciones:

La plataforma valida la CURP y el Código Postal antes de permitir el envío del formulario.

Nota: Esta tabla presenta la historia de usuario 1, en la que se describe el proceso de registro de solicitud de cita por parte del ciudadano en el portal web, incluyendo la captura de datos personales, selección de fecha y hora, y la gestión de disponibilidad por parte del Oficial del Registro Civil.

Tabla 2

Historia de usuario 2

Historia de Usuario	
Número: 2	Usuario: Ciudadano
Nombre historia: Previsualización del Costo del Trámite	
Prioridad en negocio:	Riesgo en desarrollo:
Alta	Baja
Puntos estimados: 3	Iteración asignada: 1
Programador responsable: Heidy Paola Alvarez Hernandez	
Descripción:	
El ciudadano selecciona el tipo de trámite que necesita de una lista desplegable en el portal. Al momento de seleccionarlo, la plataforma muestra de inmediato el costo correspondiente según el tabulador oficial vigente.	

El precio se actualiza automáticamente cada vez que el ciudadano cambia el tipo de trámite, sin perder los datos personales que ya había escrito en el formulario. Esto le permite saber exactamente cuánto dinero debe llevar antes de acudir a la oficina.

Observaciones:

El costo mostrado proviene directamente del catálogo de trámites administrado por el Administrador. Si el Administrador actualiza un costo, el cambio se refleja en el portal ciudadano.

Nota: Esta tabla corresponde a la historia de usuario 2, que define la funcionalidad de previsualización del costo del trámite seleccionado por el ciudadano, actualizándose de forma dinámica según el catálogo oficial vigente administrado por el Administrador.

Tabla 3

Historia de usuario 3

Historia de Usuario	
Número: 3	Usuario: Oficial del Registro Civil
Nombre historia: Libreta de Citas del Día (Agenda)	
Prioridad en negocio:	Riesgo en desarrollo:
Alta	Baja
Puntos estimados: 4	Iteración asignada: 2
Programador responsable: Heidy Paola Alvarez Hernandez	
Descripción:	
El Oficial accede a un panel digital protegido con usuario y contraseña donde puede ver la lista completa de todas las personas que tienen cita ese día y en los días siguientes. Las citas del día aparecen al inicio de la lista. Cada fila de la lista muestra: Folio de la cita, Nombre del ciudadano, CURP, Tipo de Trámite, Fecha y Hora. Esto le permite al Oficial saber con anticipación cuántos trámites se van a procesar en la jornada, de qué tipo son y tener los libros y materiales listos antes de que lleguen los ciudadanos.	

Observaciones:
Solo el personal autorizado con usuario y contraseña del Registro Civil puede acceder al panel. Los ciudadanos no tienen acceso a esta sección.

Nota: Esta tabla describe la historia de usuario 3, referente a la agenda digital del Oficial del Registro Civil, que permite consultar las citas programadas del día y días posteriores con detalle de folio, ciudadano, trámite y horario.

Tabla 4

Historia de usuario 4

Historia de Usuario	
Número: 4	Usuario: Capturista
Nombre historia: Registro de Asistencia del Solicitante	
Prioridad en negocio:	Riesgo en desarrollo:
Alta	Media
Puntos estimados: 4	Iteración asignada: 2
Programador responsable: Heidy Paola Alvarez Hernandez	
Descripción:	
Al presentarse el ciudadano en la recepción, el capturista escanea el código QR que el ciudadano recibió al agendar su cita. La plataforma valida el folio del QR y, si la cita está en estado Pendiente, la marca automáticamente como finalizada.	

El cambio de estado se refleja de forma visual en el panel, actualizando la lista. Una vez que una cita se marca como finalizada, el QR queda bloqueado y no puede volver a utilizarse para registrar otra asistencia con el mismo folio.

Observaciones:
Si el QR ya fue utilizado o el folio no existe, el sistema muestra un mensaje de error claro al capturista. solo el Oficial y el Administrador pueden revertir o cancelar una cita marcada por error.

Nota: Esta tabla presenta la historia de usuario 4, que detalla el proceso de registro de asistencia del solicitante mediante el escaneo de un código QR al momento de presentarse en ventanilla, actualizando automáticamente el estado de la cita.

Tabla 5

Historia de usuario 5

Historia de Usuario	
Número: 5	Usuario: Administrador
Nombre historia: Gestión del Catálogo de Trámites	
Prioridad en negocio:	Riesgo en desarrollo:
Alta	Baja
Puntos estimados: 5	Iteración asignada: 4
Programador responsable: Heidy Paola Alvarez Hernandez	
Descripción:	
El Administrador puede ver, crear, editar y desactivar trámites del catálogo desde su panel. Cada trámite contiene: nombre, duración en minutos, costo, documentos requeridos y estado activo o inactivo.	
Cada vez que el Administrador modifica el costo de un trámite, la plataforma genera automáticamente un registro en la bitácora con la fecha, el costo anterior, el costo nuevo y el nombre del usuario que realizó el cambio. No es posible eliminar ni desactivar un trámite que tenga citas futuras programadas en estado Pendiente.	
Observaciones:	
Los trámites desactivados dejan de aparecer en el portal ciudadano ***.	

Nota: Esta tabla corresponde a la historia de usuario 5, que describe la gestión del catálogo de trámites por parte del Administrador, incluyendo creación, edición y desactivación de trámites, así como el registro automático en bitácora de los cambios de costo.

Tabla 6

Historia de usuario 6

Historia de Usuario	
Número: 6	Usuario: Administrador
Nombre historia: Control de Acceso por Roles	
Prioridad en negocio:	Riesgo en desarrollo:
Alta	Media
Puntos estimados: 4	Iteración asignada: 2
Programador responsable: Heidy Paola Alvarez Hernandez	
Descripción:	
El Administrador gestiona las cuentas de usuario del personal interno asignando uno de tres roles: Administrador, Oficial Principal o Capturista. Cada rol tiene acceso únicamente a las funciones que le corresponden dentro de la plataforma.	
Cada vez que un usuario inicia sesión, la plataforma registra automáticamente en la bitácora su nombre, y la fecha y hora. Las contraseñas deben cumplir una política mínima de seguridad: al menos 8 caracteres, una letra mayúscula y un número.	
Observaciones:	
Un usuario sin sesión activa que intente acceder a cualquier vista interna es redirigido automáticamente a la página de inicio de sesión. El portal ciudadano no requiere login.	

Nota: Esta tabla presenta la historia de usuario 6, que define el control de acceso por roles dentro de la plataforma, permitiendo al Administrador gestionar cuentas de usuario con niveles de acceso diferenciados y registro de sesiones en bitácora.

Tabla 7

Historia de usuario 7

Historia de Usuario	
Número: 7	Usuario: Oficial del Registro Civil
Nombre historia: Registro Automático de Auditoría (Bitácora)	
Prioridad en negocio:	Riesgo en desarrollo:
Alta	Baja
Puntos estimados: 3	Iteración asignada: 4
Programador responsable: Heidy Paola Alvarez Hernandez	
Descripción:	
El Oficial puede consultar un log completo de todas las acciones realizadas en la plataforma. Cada vez que se modifica un dato crítico (costo de trámite, estado de una cita, creación o cambio de usuario), en la plataforma genera automáticamente un registro en la bitácora sin que nadie tenga que hacerlo manualmente.	
La bitácora puede filtrarse por usuario, rango de fechas y tipo de acción. Ningún rol, incluyendo el Administrador, puede editar ni eliminar registros de la bitácora.	
Observaciones:	
La bitácora es el único registro de auditoría de la plataforma. Esto ayuda a que cualquier irregularidad quede evidenciada sin posibilidad de ser alterada.	

Nota: Esta tabla describe la historia de usuario 7, que establece el sistema de registro automático de auditoría (bitácora), donde se documentan todas las acciones críticas realizadas en la plataforma de forma inmutable y consultable con filtros

Tabla 8

Historia de usuario 8

Historia de Usuario	
Número: 8	Usuario: Oficial
Nombre historia: Registro de Pago en Caja Simulada	
Prioridad en negocio:	Riesgo en desarrollo:
Alta	Baja
Puntos estimados: 5	Iteración asignada: 3
Programador responsable: Heidy Paola Alvarez Hernandez	
Descripción:	
El Oficial registra el pago del ciudadano al momento de su trámite. Al registrar el pago, la cita cambia automáticamente a estado Finalizado. Si el Capturista escaneó un QR por error, solo el Oficial o el Administrador pueden cancelar esa acción para que no afecte el corte de caja.	
Al cierre del día, la plataforma genera automáticamente un corte de caja con el total recaudado desglosado por tipo de trámite. El panel principal muestra únicamente el monto recaudado en el mes en curso, y existe una sección de historial donde se pueden consultar los cortes de todos los meses anteriores. La caja es simulada y no se conecta con ninguna acción de pago real por media de la plataforma web.	
Observaciones:	
Los cortes de caja generados son inmutables. Una vez cerrado el día, el registro no puede modificarse ni eliminarse.	

Nota: Esta tabla corresponde a la historia de usuario 8, que detalla el registro de pago en caja simulada, incluyendo el corte de caja diario automático, el historial mensual de recaudación y la restricción de modificación de los registros generados.

Evidencia de la recopilación de las historias de usuarios

FALTAN ANEXARLAS

Requisitos funcionales

Conforme a las necesidades presentadas en las historias de usuario se hizo el levantamiento de los requisitos funcionales presentados la tabla 9, 10, 11, 12 y 13

Tabla 9

Requisitos funcionales del portal ciudadano

ID	Nombre	Descripción	HU	Prioridad
RF-01	Selección de trámite	El ciudadano elige el trámite de una lista ordenada. Al seleccionar, la plataforma muestra de inmediato el costo actualizado y los documentos requeridos sin recargar la página.	HU-2	Alta
RF-02	Captura de datos personales	El ciudadano ingresa nombre completo, CURP, Código Postal, teléfono y correo electrónico en el formulario de agendado.	HU-1	Alta
RF-03	Validación de CURP en tiempo real	La plataforma valida la CURP mientras el ciudadano la escribe, mostrando indicador de error si no cumple el formato exacto de 18 caracteres alfanuméricos.	HU-1	Alta
RF-04	Selección de fecha y hora disponible	El ciudadano elige fecha en un calendario interactivo. La plataforma muestra solo los horarios disponibles según la duración del trámite seleccionado.	HU-1	Alta
RF-05	Prevención de empalmes concurrentes	Si dos ciudadanos intentan agendar el mismo horario simultáneamente, la plataforma acepta solo al primero y solicita al segundo que elija otro horario disponible.	HU-1	Alta
RF-06	Generación de código QR	Al confirmar la cita, la plataforma genera automáticamente un código QR único vinculado al folio de la cita. El ciudadano puede descargarlo.	HU-4	Alta
RF-07	Confirmación y resumen de cita	El ciudadano ve un resumen completo con: folio, nombre, trámite, fecha, hora, costo y el código QR descargable.	HU-1	Alta

Nota: Esta tabla lista los requisitos funcionales del portal ciudadano (RF-01 al RF-07), que abarcan desde la selección del trámite y captura de datos personales hasta la generación del código QR y confirmación de cita.

Tabla 10

Requisitos funcionales del panel del Oficial

ID	Nombre	Descripción	HU	Prioridad
RF-08	Agenda diaria de citas	El Oficial visualiza la lista de todas las citas del día y días siguientes, ordenadas por hora, con las de hoy al inicio de la pantalla.	HU-3	Alta
RF-09	Modificación de disponibilidad horarios	El Oficial puede bloquear o habilitar días y horarios desde el panel de calendario, de forma semanal o inmediata ante cualquier contratiempo.	HU-1	Alta
RF-10	Registro de pago en caja	El Oficial registra el pago del ciudadano al momento del trámite, cambiando el estado de la cita de ASISTIDA a PAGADA.	HU-8	Alta
RF-11	Corte de caja diario	Al cierre del día la plataforma genera automáticamente el corte de caja con el total recaudado desglosado por tipo de trámite.	HU-8	Alta

Nota: Esta tabla presenta los requisitos funcionales del panel del Oficial (RF-08 al RF-11), que incluyen la agenda diaria de citas, la modificación de disponibilidad de horarios, el registro de pago en caja y la generación del corte de caja diario.

Tabla 11

Requisitos funcionales del módulo Capturista

ID	Nombre	Descripción	HU	Prioridad
RF-12	Escaneo de código QR	El Capturista escanea el QR del ciudadano al presentarse en ventanilla para registrar su asistencia y cambiar el estado de la cita a ASISTIDA.	HU-4	Alta
RF-13	Actualización en tiempo real del estado	Al escanear el QR, el estado de la cita cambia visualmente de PENDIENTE a ASISTIDA en el panel sin necesidad de recargar la página.	HU-4	Alta

Nota: Esta tabla describe los requisitos funcionales del módulo Capturista (RF-12 y RF-13), centrados en el escaneo del código QR del ciudadano y la actualización en tiempo real del estado de la cita en el panel.

Tabla 12

Requisitos funcionales del panel Administrador

ID	Nombre	Descripción	HU	Prioridad
RF-14	Gestión del catálogo de trámites	El Administrador crea, edita, activa y desactiva trámites con: nombre, duración, costo, documentos requeridos y estado. No se puede desactivar un trámite con citas futuras pendientes.	HU-5	Alta
RF-15	Historial de cambios de costo	Cada cambio de costo genera automáticamente un registro con: fecha, costo anterior, costo nuevo y usuario responsable. El historial es inmutable.	HU-5	Alta
RF-16	Gestión de usuarios y roles	El Administrador crea, edita y desactiva cuentas de usuario asignando el rol correspondiente: Administrador, Oficial o Capturista.	HU-6	Alta
RF-17	Consulta de bitácora de auditoría	El Administrador consulta el log completo de la plataforma con filtros por usuario, rango de fechas y tipo de acción. La bitácora no puede editarse ni eliminarse.	HU-7	Alta
RF-18	Panel de caja mensual e historial	El panel muestra el monto total recaudado en el mes en curso y permite consultar el historial de cortes de meses anteriores.	HU-8	Media
RF-19	Cancelación de citas	El Administrador puede cancelar citas en estado PENDIENTE o ASISTIDA. La cancelación queda registrada en bitácora con motivo, usuario y fecha.	HU-8	Alta

Nota: Esta tabla presenta los requisitos funcionales del panel Administrador (RF-14 al RF-19), que comprenden la gestión del catálogo de trámites, historial de cambios de costo, gestión de usuarios y roles, consulta de bitácora, panel de caja y cancelación de citas.

Tabla 13

Requisitos funcionales de autenticación y control de acceso

ID	Nombre	Descripción	HU	Prioridad
RF-20	Login de usuarios internos	El personal accede a la plataforma con usuario y contraseña. El portal ciudadano no requiere login. La sesión expira tras un tiempo de inactividad configurable.	HU-6	Alta
RF-21	Control de acceso por rol	Cada rol accede únicamente a las vistas y funciones que le corresponden. Si el rol no coincide con la vista, la plataforma redirige automáticamente.	HU-6	Alta
RF-22	Registro de accesos en bitácora	Cada inicio y cierre de sesión queda registrado automáticamente con: IP del cliente, usuario, fecha y hora exactas.	HU-6	Alta
RF-23	Política de contraseñas	Las contraseñas deben tener mínimo 8 caracteres, al menos una letra mayúscula y un número. La plataforma rechaza contraseñas que no cumplan esta política.	HU-6	Alta

Nota: Esta tabla detalla los requisitos funcionales de autenticación y control de acceso (RF-20 al RF-23), que incluyen el inicio de sesión del personal interno, el control de acceso por rol, el registro de accesos en bitácora y la política de contraseñas.

Requisitos no funcionales

A continuación, se presentan los requisitos no funcionales en la tabla 14

Tabla 14

Requisitos no funcionales

ID	Nombre	¿Qué debe cumplir la plataforma?
RNF-01	Contraseñas seguras	Las contraseñas del personal deben tener al menos 8 caracteres, una letra mayúscula y un número.
RNF-02	Acceso restringido	Las páginas del panel interno solo pueden verlas quienes hayan iniciado sesión. Si alguien intenta entrar sin sesión, la plataforma lo manda al login.
RNF-03	Contraseñas cifradas	Las contraseñas nunca se guardan como texto. La plataforma las convierte en un código cifrado que nadie puede leer.
RNF-04	Bitácora intocable	El registro de acciones de la plataforma no puede ser modificado ni borrado por nadie, ni el administrador.
RNF-05	Funciona en celular	El portal donde el ciudadano agenda su cita debe verse bien y funcionar en celular, tablet y computadora, como aspecto responsivo.
RNF-06	Errores claros	Cuando el ciudadano comete un error, la plataforma debe decirle exactamente qué está mal y cómo corregirlo, no un mensaje genérico como 'Error'.
RNF-07	Colores por estado	En el panel del personal, cada cita tiene un color: amarillo (pendiente), verde (asistida), morado (finalizada), rojo (cancelada).
RNF-08	Páginas rápidas	Cualquier página de la plataforma debe cargar en 3 segundos con una conexión normal a internet.

Nota: Esta tabla presenta los requisitos no funcionales de la plataforma (RNF-01 al RNF-08), que establecen criterios de seguridad, rendimiento, usabilidad y disponibilidad que la plataforma debe cumplir de forma transversal.

Elección arquitectura para el software

Se eligió una arquitectura monolítica de tres capas basada en el patrón Modelo-Vista-Controlador (MVC) que Django implementa de forma nativa (conocido como MVT: Model-View-Template).

Esta arquitectura consiste en una única aplicación que integra todos sus componentes en un solo despliegue. Se organiza en tres capas claramente separadas:

Capa de Presentación (Vista/Template): Es la responsable de la interfaz de usuario. Aquí se gestiona todo lo que el usuario ve e interactúa, tanto en el portal público como en las interfaces internas de la plataforma.

Capa de Lógica de Negocio (Controlador): Contiene las reglas y procesos del negocio. - Es donde se implementan las validaciones, cálculos, flujos de trabajo y decisiones de la plataforma (por ejemplo, control de horarios, generación de códigos QR y corte de caja).

Capa de Datos (Modelo): Se encarga de la persistencia y recuperación de la información. Gestiona las interacciones con la base de datos, definiendo las estructuras de las tablas y las consultas necesarias.

La razón principal por la cual se eligió esta arquitectura fue que se ajusta perfectamente al tamaño y la naturaleza del proyecto. La plataforma está diseñada para trabajar en una sola oficina del Registro Civil, con tres roles de usuario internos y un portal público. No tenía sentido implementar una arquitectura distribuida o de microservicios orientada a plataformas de gran escala que deben soportar millones de usuarios simultáneos.

Django facilita de manera natural esta separación de responsabilidades, sin necesidad de configuraciones complejas. Esto permitió que el equipo se concentrara en resolver los requerimientos específicos del negocio, en lugar de invertir tiempo en la infraestructura o en la integración de múltiples componentes.

Adicionalmente, se valoró la facilidad de despliegue. Al tratarse de una aplicación monolítica, su instalación y puesta en producción en el servidor de la oficina es un proceso sencillo y directo, lo cual resulta especialmente adecuado en el contexto de una entidad gubernamental municipal con recursos tecnológicos limitados.

Esta arquitectura favorece un código ordenado y mantenible. Gracias a la clara separación de las tres capas, es fácil identificar dónde reside cada funcionalidad (lo que el usuario ve, las reglas del negocio y la información almacenada), lo que agiliza la detección de errores, la realización de modificaciones y la incorporación de nuevas funcionalidades sin comprometer la plataforma existente

Creación de base de datos

Para llegar a este modelado, primero se realizó un análisis detallado de las historias de usuario con el objetivo de identificar qué información necesita guardar el sistema, cómo se relaciona esa información entre sí y qué restricciones debe cumplir cada dato para garantizar la integridad del sistema.

Del análisis de las historias de usuario se identificaron las entidades principales. La Historia de Usuario 1 dejó claro que el sistema necesita almacenar los datos del ciudadano al momento de agendar, incluyendo su CURP con exactamente 18 caracteres, su Código Postal de 5 dígitos, la fecha, la hora y el estado de la cita. La Historia de Usuario 2 indicó que los trámites deben tener un costo asociado que pueda consultarse en tiempo real. Las Historias de Usuario 3 y 4

revelaron la necesidad de registrar qué usuario del personal atendió cada cita y de almacenar un código QR único vinculado a cada folio. La Historia de Usuario 5 mostró que los trámites deben organizarse por sección o categoría y que cada uno tiene una duración, un costo y documentos requeridos. La Historia de Usuario 6 evidenció la necesidad de una tabla de usuarios con un campo de rol diferenciado. La Historia de Usuario 7 dejó en claro que todas las acciones críticas deben quedar registradas en una tabla de auditoría inmutable con la acción realizada, la descripción, la fecha, la hora. Finalmente, la Historia de Usuario 8 indicó que cada cita puede tener un pago asociado con el monto cobrado, la fecha del pago y el cajero que lo registró.

Con ese análisis completo, se determinaron las relaciones entre las entidades. Una sección contiene muchos trámites, pero cada trámite pertenece a una sola sección. Un trámite puede tener muchas citas asociadas, pero cada cita pertenece a un único trámite. Cada cita genera exactamente un pago, estableciendo una relación de uno a uno entre ambas tablas. Los usuarios del personal se relacionan con las citas para registrar quién las atendió y con los pagos para registrar quién cobró. La bitácora se relaciona con los usuarios para identificar el responsable de cada acción, aunque ese campo puede quedar vacío cuando la acción la genera el sistema de forma automática, como cuando un ciudadano agenda una cita sin haber iniciado sesión.

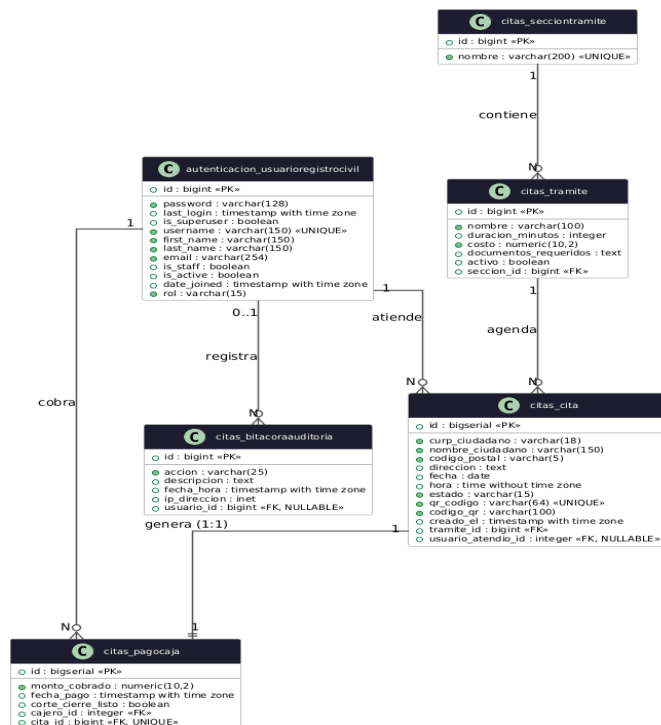
Una vez completado el análisis, se utilizó PlantUML como herramienta de representación para documentar de forma clara y estructurada las tablas del sistema, sus campos, sus tipos de datos y las relaciones entre ellas.

El modelado se organizó en dos grupos. El primer grupo pertenece a la aplicación autenticación y contiene la tabla autenticacion_usuarioregistrocivil, que representa a los usuarios internos del sistema con su campo de rol diferenciado entre Administrador, Oficial y Capturista. El segundo

grupo pertenece a la aplicación citas y contiene las cinco tablas restantes: citas_secciontramite, que agrupa los trámites por categoría; citas_tramite, que almacena el catálogo de trámites con su duración, costo y documentos requeridos; citas_cita, que es la tabla central del sistema y guarda todos los datos del ciudadano junto con el estado de la cita y el código QR único vinculado al folio; citas_pagocaja, que registra el pago de cada cita en una relación de uno a uno garantizando que ninguna cita pueda tener más de un pago asociado; y citas_bitacoraauditoria, que guarda el registro inmutable de todas las acciones críticas realizadas en el sistema. En cada tabla se especificaron las claves primarias, las claves foráneas que establecen las relaciones entre tablas y los campos con restricción de unicidad. El diagrama generado con PlantUML constituyó la guía de referencia para la implementación posterior de la base de datos en PostgreSQL, se muestra el modelado en la figura 2.

Figura 2

Modelado de base de datos



Nota. Modelado de la base de datos del Sistema de Gestión de Citas del Registro Civil Municipal elaborado en PlantUML. Elaboración propia (2026).

La separación en dos modelos de datos es una decisión de diseño basada en la diferencia de responsabilidades entre ambos grupos de entidades.

Por un lado, todo lo relacionado con los usuarios y el acceso a la plataforma tiene una naturaleza distinta al resto del proyecto. El modelo `autenticacion_usuarioregistrocivil` necesita estar configurado en Django antes que cualquier otra cosa, ya que reemplaza al usuario estándar del framework y otras entidades de la plataforma dependen de él como referencia. Si este modelo viviera junto con los demás, cualquier cambio en la configuración de autenticación implicaría tocar la misma aplicación donde están las citas, los trámites y los pagos, lo que aumenta el riesgo de errores y complica el mantenimiento.

Por otro lado, las entidades relacionadas con las citas, los trámites, los pagos y la bitácora representan el dominio principal del negocio. Son las que cambian con más frecuencia durante el desarrollo, las que concentran la mayor parte de la lógica de la plataforma y las que tienen relaciones más complejas entre sí. Mantenerlas en su propia aplicación permite trabajar sobre ellas de forma independiente sin afectar la configuración de acceso a la plataforma.

Diseño de las tarjetas CRC

Tabla 15

Usuario

UsuarioRegistroCivil Modelo —autenticación Capa: Datos	
RESPONSABILIDADES	COLABORADORES
• Heredar los campos base de AbstractUser de Django (usuario, contraseña, email, etc.) • Agregar el campo rol: ADMINISTRADOR, OFICIAL o CAPTURISTA (default: CAPTURISTA) • Proveer get_rol_display() para mostrar el nombre del rol en la interfaz • Reemplazar al Usuario estándar mediante AUTH_USER_MODEL en settings.py	→ Abstracción de Usuario de Django (lo hereda) → Cita (usuario_atendio_id referencia este modelo) → PagoCaja (cajero_id referencia este modelo) → BitacoraAuditoria (usuario_id referencia este modelo)

Nota: Esta tabla describe el modelo UsuarioRegistroCivil de la capa de datos en la aplicación de autenticación, el cual extiende el usuario estándar de Django añadiendo el campo de rol (Administrador, Oficial o Capturista) y sirve como referencia para citas, pagos y la bitácora.

Tabla 16

Login

Login Lógica de acceso —autenticación Capa: Lógica de Negocio	
RESPONSABILIDADES	COLABORADORES
• Autenticar al usuario verificando usuario y contraseña • Validar política de contraseña: mínimo 8 caracteres, una mayúscula y un número • Iniciar y cerrar la sesión del usuario • Redirigir al panel correcto según el rol tras iniciar sesión • Registrar en bitácora cada acceso con IP, usuario, fecha y hora • Proteger vistas internas rechazando peticiones sin sesión activa	→ UsuarioRegistroCivil (verifica credenciales y obtiene el rol) → BitacoraAuditoria (registra cada acceso)

Nota: Esta tabla presenta la clase Login de la capa de lógica de negocio en autenticación, responsable de verificar credenciales, aplicar la política de contraseñas, gestionar sesiones, redirigir según rol y registrar cada acceso en la bitácora.

Tabla 17

Sección trámite

SeccionTramite Modelo —citas Capa: Datos	
RESPONSABILIDADES	COLABORADORES
• Almacenar el nombre de la sección o categoría (VARCHAR 200, UNIQUE) • Agrupar trámites por tipo para facilitar la navegación del ciudadano	→ Tramite (una sección contiene muchos trámites)

Nota: Esta tabla agrupa los modelos de la capa de datos de la aplicación citas: SeccionTramite (categorías), Tramite (catálogo), Cita (registro central de citas), PagoCaja (pagos) y BitacoraAuditoria (auditoría inmutable), estableciendo las relaciones entre ellos.

Tabla 18

Tramite

Tramite Modelo —citas Capa: Datos	
RESPONSABILIDADES	COLABORADORES
• Almacenar nombre, duración en minutos, costo, documentos requeridos y estado activo/inactivo • Pertenecer a una SeccionTramite mediante llave foránea • Exponer la duración en formato legible (ej: 1h 30min)	→ SeccionTramite (pertenece a una sección) → Cita (cada cita referencia un trámite) → Tramite (gestiona su creación y modificación)

Nota: Esta tabla describe la clase Cita de la capa de lógica de negocio, que gestiona el flujo completo de una cita: validaciones de CURP y código postal, creación atómica, generación de QR, cambios de estado y registro en bitácora.

Tabla 19

Cita

Cita Modelo central —citas Capa: Datos	
RESPONSABILIDADES	COLABORADORES
• Almacenar CURP (CHAR 18), nombre ciudadano (VARCHAR 150) y código postal (CHAR 5) • Guardar fecha, hora y estado actual (VARCHAR 15): PENDIENTE, ASISTIDA, PAGADA, FINALIZADA, CANCELADA • Almacenar código QR único (VARCHAR 64, UNIQUE) e imagen del QR (ImageField) • Registrar qué usuario atendió la cita (FK nullable a UsuarioRegistroCivil) • Guardar la fecha y hora de creación del registro (DATETIME) • Forzar que la CURP siempre se almacene en mayúsculas	→ Tramite (cada cita pertenece a un trámite) → UsuarioRegistroCivil (registra quién atendió) → PagoCaja (relación 1:1, genera un pago) → BitacoraAuditoria (sus cambios se registran automáticamente) → Cita (la crea, consulta y gestiona su flujo de estados)

Nota: Esta tabla presenta la clase Calendario de la capa de lógica de negocio, encargada de verificar disponibilidad de horarios, detectar empalmes mediante bloqueo de registros, generar la lista de horarios libres y permitir al Oficial bloquear franjas.

Tabla 20

Pago caja

PagoCaja Modelo —citas Capa: Datos	
RESPONSABILIDADES	COLABORADORES
• Almacenar monto cobrado (DECIMAL 10,2), fecha y hora del pago (DATETIME) • Relacionarse de forma 1:1 con una Cita (FK UNIQUE) • Registrar al cajero que realizó el cobro (FK a UsuarioRegistroCivil) • Indicar mediante campo booleano si el pago ya fue incluido en el corte del día	→ Cita (relación 1:1, cada pago pertenece a una sola cita) → UsuarioRegistroCivil (registra al cajero responsable) → Caja (lo crea y lo incluye en el corte diario)

Nota: Esta tabla describe la clase QR de la capa de lógica de negocio, responsable de generar el código QR al crear una cita, guardarlo en el modelo y validarlo al momento en que el Capturista lo escanea para registrar la asistencia del ciudadano.

Tabla 21

Bitácora auditoria

BitacoraAuditoria Modelo de auditoría —citas Capa: Datos	
RESPONSABILIDADES	COLABORADORES
• Almacenar acción (VARCHAR 25), descripción (TEXT), fecha y hora (DATETIME) e IP (INET) • Relacionarse con el usuario responsable (FK nullable) • Ser completamente inmutable: bloquear cualquier intento de modificación o borrado	→ UsuarioRegistroCivil (referencia quién realizó la acción) → Bitacora (la crea automáticamente desde la lógica de negocio)

Nota: Esta tabla describe la clase Caja de la capa de lógica de negocio, que gestiona el registro de pagos, verifica roles para cancelaciones, genera el corte de caja diario desglosado por trámite y expone el resumen mensual e historial al panel.

Tabla 22

Cita

Cita Lógica principal —citas Capa: Lógica de Negocio	
RESPONSABILIDADES	COLABORADORES
• Validar que la CURP tenga exactamente 18 caracteres alfanuméricos • Validar que el Código Postal tenga exactamente 5 dígitos numéricos • Verificar disponibilidad del horario antes de crear la cita • Crear la cita dentro de una transacción atómica para evitar datos incompletos • Coordinar la generación del QR tras confirmar la cita • Registrar en bitácora cada cita creada • Listar citas del día ordenadas por hora para el panel del Oficial • Gestionar el flujo de estados: PENDIENTE → ASISTIDA → PAGADA → FINALIZADA • Permitir cancelaciones solo a usuarios con rol OFICIAL o ADMINISTRADOR	→ Cita (crea y consulta registros) → Tramite (obtiene duración y costo) → Calendario (verifica disponibilidad) → QR (genera código e imagen QR) → Bitacora (registra cada acción) → UsuarioRegistroCivil (verifica rol para cancelaciones)

Nota: Esta tabla describe la clase Caja de la capa de lógica de negocio, que gestiona el registro de pagos, verifica roles para cancelaciones, genera el corte de caja diario desglosado por trámite y expone el resumen mensual e historial al panel.

Tabla 23

Calendario

Calendario Control de horarios —citas Capa: Lógica de Negocio	
RESPONSABILIDADES	COLABORADORES
• Verificar que un horario esté libre considerando la duración del trámite • Detectar empalmes con bloqueo de registros a nivel de base de datos • Generar la lista de horarios disponibles para una fecha y trámite específicos • Respetar el horario de atención de la oficina • Permitir al Oficial bloquear días o franjas horarias desde el panel	→ Cita (consulta citas existentes para detectar empalmes) → Tramite (necesita la duración para calcular disponibilidad)

Nota: Esta tabla presenta la clase Bitacora de la capa de lógica de negocio, encargada de crear registros inmutables en BitacoraAuditoria por cada acción crítica del sistema, y de proveer la consulta filtrada por usuario, fecha y tipo de acción.

Tabla 24

Generación y validación de QR

QR Generación y validación de QR —citas Capa: Lógica de Negocio	
RESPONSABILIDADES	COLABORADORES
• Generar el código QR a partir del código único de la cita • Guardar la imagen del QR en el campo ImageField del modelo Cita • Validar el código escaneado por el Capturista al llegar el ciudadano • Verificar que la cita esté en estado PENDIENTE antes de marcarla ASISTIDA • Rechazar códigos ya utilizados o inexistentes con mensaje de error claro	→ Cita (guarda el QR y verifica el estado) → Cita (lo llama al crear la cita y al escanear el QR)

Nota: Esta tabla describe el modelo de datos adicional de la capa de datos, relacionado con la gestión del catálogo de trámites y sus secciones, como complemento a las tablas principales de la aplicación citas.

Tabla 25

Tramite

Tramite Gestión del catálogo —citas Capa: Lógica de Negocio	
RESPONSABILIDADES	COLABORADORES
• Listar trámites activos con costo actualizado para el portal ciudadano • Permitir al Administrador crear, editar, activar y desactivar trámites • Verificar que un trámite no tenga citas futuras antes de desactivarlo • Registrar en bitácora cada cambio de costo con fecha, valor anterior, valor nuevo y usuario • Proveer el costo al portal cuando el ciudadano cambia de trámite sin recargar la página	→ Tramite (gestiona sus registros) → SeccionTramite (agrupa los trámites por categoría) → Cita (consulta citas futuras antes de desactivar) → Bitacora (registra cambios en el catálogo)

Nota: Esta tabla presenta un componente adicional de la lógica de negocio de la aplicación citas, que complementa las responsabilidades de gestión de citas y colabora con los modelos principales del sistema.

Tabla 26

Caja

Caja Control de pagos—citas Capa: Lógica de Negocio	
RESPONSABILIDADES	COLABORADORES
• Registrar el pago de una cita al momento de su atención • Verificar que solo el Oficial o Administrador puedan cancelar un pago • Marcar corte_cierre_listo en True al incluir el pago en el corte del día • Generar el corte de caja al cierre del día desglosado por tipo de trámite • Proveer al panel el monto recaudado en el mes en curso e historial de meses anteriores	→ PagoCaja (crea y consulta registros de pago) → Cita (obtiene el monto del trámite al registrar el pago) → UsuarioRegistroCivil (verifica rol para cancelaciones) → Bitacora (registra cada pago y cancelación)

Nota: Esta tabla describe un módulo adicional de la capa de lógica de negocio, que apoya funciones específicas de auditoría o control de acceso dentro de la aplicación.

Tabla 27

Bitácora

Bitacora Auditoría—Roles Capa: Lógica de Negocio	
RESPONSABILIDADES	COLABORADORES
• Crear registros inmutables en BitacoraAuditoria por cada acción crítica • Registrar: acción, descripción, usuario responsable, fecha y hora • Nunca modificar ni eliminar registros existentes • Permitir consulta con filtros por usuario, fecha y tipo de acción	→ BitacoraAuditoria (crea sus registros)

Nota: Esta tabla presenta el último componente definido en el documento, correspondiente a una clase o módulo de apoyo dentro de la arquitectura de la plataforma del Registro Civil.

Contribuciones realizadas

Productos del proyecto

Conclusiones

Recomendaciones

Referencias

Beck, K., y Andres, C. (2004). Extreme programming explained: Embrace change (2.^a ed.). Addison-Wesley Professional.

Briano, C. A. (2023). Compilación de apuntes sobre conceptos fundamentales de la ingeniería de software (2.^a ed.). Universidad de Buenos Aires, Facultad de Ciencias Económicas.
http://bibliotecadigital.econ.uba.ar/download/libros/Briano_compilacion_apuntes.pdf

Django Software Foundation. (2024). Django: el framework web para perfeccionistas con plazos (Versión 5.x) [Software]. <https://docs.djangoproject.com>

Gobierno de México. (2024a). Guía de identidad gráfica digital (pp. 26-50). Secretaría de Gobernación.
https://www.gob.mx/cms/uploads/attachment/file/993332/Gu_a_de_identidad_gr_fica_P.26-50.pdf

Gobierno de México. (2024b). Guía gráfica GOB.MX versión 2.
<https://www.gob.mx/guias/grafica/v2/>

Gobierno de México. (2024c). Guía gráfica GOB.MX versión 3: Botones.
<https://www.gob.mx/guias/grafica/v3/botones.html>

Organización Internacional de Normalización. (2018). ISO 9241-11:2018: Ergonomía de la interacción humano-sistema — Parte 11: Usabilidad: Definiciones y conceptos. ISO.
<https://www.iso.org/standard/63500.html>

Organización Internacional de Normalización y Comisión Electrotécnica Internacional. (2024). ISO/IEC 18004:2024: Tecnología de la información Técnicas automáticas de identificación y captura de datos Especificación de simbología de código de barras de código QR. ISO.

Letelier, P., y Penadés, M. C. (2006). Metodologías ágiles para el desarrollo de software: eXtreme Programming (XP). *Técnica Administrativa*, 5(26).
<https://www.cyta.com.ar/ta/article.php?id=050201>

Marcotte, E. (2015). *Diseño web responsivo* (2.^a ed.). A Book Apart.

Montilva C., J., y Barrios A., J. (2022). *Ingeniería del software: Un enfoque basado en procesos*. (Premio "Juan Manuel Cagigal" al mejor libro-texto del año 2022, Academia Nacional de Ingeniería y Habitat de Venezuela). <https://www.researchgate.net/publication/368778144>

Morales, R. (2021). *Desarrollo de plataformas web con tecnologías de código abierto*. Editorial Académica Española.

Pressman, R. S., y Maxim, B. R. (2014). *Software engineering: A practitioner's approach* (8.^a ed.). McGraw-Hill Education.

PlantUML. (2025). *Guía de referencia del lenguaje PlantUML* (Versión 1.2025.0).
<https://plantuml.com/>

PostgreSQL Global Development Group. (2024). *Documentación de PostgreSQL* (Versión 16).
<https://www.postgresql.org/docs/16/>

Python Software Foundation. (2024). *Python* (Versión 3.12) [Lenguaje de programación].
<https://www.python.org/>

Render. (2024). *Documentación de Render: Servicios web*. Render Inc.
<https://render.com/docs/web-services>

World Wide Web Consortium. (2018). *Pautas de Accesibilidad para el Contenido Web (WCAG)* 2.1. Recomendación W3C. <https://www.w3.org/TR/WCAG21/>